

McKenna Group S.A.S. · Bogotá, Colombia

Manual de Usuario

Agente de Automatización Empresarial

Hugo García

Guía completa de operación, comandos, arquitectura técnica y flujos de trabajo
del sistema de automatización empresarial de McKenna Group · v3.1 · 24 de May de 2026

97%	24/7	30	12+
Automatización	Disponibilidad	Herramientas IA	Módulos activos

ÍNDICE

Tabla de Contenidos

01 Introducción y Visión General

- Qué es Hugo García
- Canales de comunicación
- Requisitos para operar

02 Arquitectura del Sistema

- 3 procesos independientes
- Integraciones externas
- Stack tecnológico

03 Módulos Funcionales

- MOD-01 a MOD-10 descritos

04 Guía de Comandos — Grupo de Contabilidad

- Confirmación de pagos
- Preventa MercadoLibre
- Control IA/Humano
- Respuestas directas
- Postventa MeLi (grupo dedicado)
- Aprobación de facturas

05 Flujo de Atención al Cliente WhatsApp

- Primer contacto
- Consulta de productos
- Cotización paso a paso
- Envío de comprobante
- Facturación y despacho

06 Preventa MercadoLibre

- Cómo funciona la IA
- Fichas técnicas en Google Sheets
- Respuesta manual del operador
- Aprendizaje continuo

07 Sincronización de Inventario

- Principio de stock
- MeLi ↔ página web (API)
- Comandos de sync

08 Facturación SIIGO y Documentos Fiscales

- Flujo A: Compra materias primas (Gmail → XML → SIIGO)
- Flujo B: Facturación de venta a clientes
- Protocolo de carga en SIIGO
- Tabla de conversión de unidades

09 Panel Web y Menú CLI

- Panel en navegador
- Menú interactivo CLI

10 Monitor de Alertas Automáticas

- Alertas programadas
- Reportes automáticos
- Backup nocturno

11 Glosario Técnico y Preguntas Frecuentes

- Términos clave
- FAQ operadores
- Solución de problemas

12 Generación de Contenido Científico y Web

- Knowledge Agent: PubMed + ArXiv + Scrapling
- Pipeline multimedia Facebook
- Publicación automática en WordPress
- Scripts de generación masiva

13 Nuevas Skills (Herramientas Autónomas)

- Tool calling con Claude en core.py
- Ejemplos: precios, catálogo PDF, Facebook, guías web
- Stock hacia tienda: sincronizar_productos_pagina_web
- Registro en todas_las_herramientas

14 Observabilidad, auditoría de scripts y backup

- request_id y AGENTE_LOG_JSON
- Restricción de parchear_funcion / ejecutar_script
- Manifiesto, cron, auditar_scripts
- Backup nocturno, GitHub, GRUPO_ALERTAS_SISTEMAS_WA

15 Metodología Agéntica McKenna

- Orquestador y subagentes
- Memoria local tipo Engram
- Skills bajo demanda
- Ecosistema Gentleman
- Contratos, smoke tests y CI backend

16 Refactorización Agéntica 2026-05-24: AgentRun + Tricap Memory

- AgentRun — máquina de estados determinista (reemplaza bucle monolítico)
- LLMRouter — multi-proveedor Claude / Gemini / Ollama con fallback automático
- ToolDispatcher — ejecución con registro episódico
- CheckpointStore — checkpoints SQLite por iteración
- Tricap Memory — Working + Episodic + Semantic + Compressor
- Integración con core.py y validación (96/96 tests)

SECCIÓN 01

Introducción y Visión General

Hugo García es el agente de automatización empresarial de McKenna Group S.A.S., una empresa colombiana especializada en materias primas farmacéuticas y cosméticas con sede en Bogotá. El agente opera como un asistente ejecutivo digital disponible 24 horas al día, 7 días a la semana, gestionando de forma autónoma los principales procesos comerciales y operativos de la empresa: atención a clientes por WhatsApp, respuestas a compradores en MercadoLibre, sincronización de inventario, facturación electrónica en SIIGO ERP, confirmación de pagos y generación de reportes.

- **Objetivo principal:** Eliminar el trabajo repetitivo del equipo humano en tareas que pueden automatizarse, permitiendo que el personal de ventas y contabilidad se concentre en actividades de mayor valor: relaciones con clientes estratégicos, negociaciones especiales y crecimiento del negocio.

1.1 Canales de Comunicación

El agente está conectado simultáneamente a los siguientes canales, procesando mensajes en tiempo real:

Canal	Puerto / Servicio	Responsabilidad
WhatsApp Business	bot-mckenna (Node) :3000 → Flask :8081 /whatsapp	Atención clientes, pagos, comandos por grupo (contabilidad, pedidos web, etc.)
MercadoLibre	Webhook API · Flask :8080	Preventa (Gemini + ficha), órdenes pagadas (sync stock), posventa (API mensajes v2, adjuntos PDF/imagen)
Página web / tienda	Flask PAGINA_WEB/site + API propia	Ventas web; stock se alinea con MeLi vía WEB_API_URL / sincronizar_productos_pagina_web
SIIGO ERP	API REST HTTPS	Facturación electrónica, facturas de venta y de compra
Google Sheets	gsread API	Catálogo, fichas técnicas (col I), precios
Gmail	OAuth 2.0 · IMAP	Facturas electrónicas de proveedores (XML DIAN)
Google Drive	Drive API v3	Backups nocturnos automáticos de datos críticos

1.2 Roles y Acceso

Rol	Canal de acceso	Capacidades
Cliente final	WhatsApp	Consultar productos, solicitar cotización, enviar comprobante de pago
Operador (grupo contabilidad)	WhatsApp – grupo contabilidad	Confirmar/rechazar pagos, pausar IA, facturas compra, comandos inventario
Operador (grupo preventa MeLi)	WhatsApp – GRUPO_PREVENTA_WA	Responder preguntas pendientes de publicaciones (comandos resp ...)
Operador (grupo posventa MeLi)	WhatsApp – GRUPO_POSTVENTA_WA	Alertas de mensajes post-compra; comando posventa <código>: ...; aprobar borradores IA con hugo dale ok
Administrador	CLI / endpoints /sync/*	Sincronizaciones manuales, ajustes de stock, reportes, backups

Rol	Canal de acceso	Capacidades
Sistema (IA)	Interno · automático	Todo lo anterior de forma autónoma según reglas configuradas

Importante: El agente opera con el número de WhatsApp Business de McKenna Group. Los operadores del grupo de contabilidad tienen acceso a comandos especiales que el agente escucha exclusivamente en ese grupo. Los mensajes de clientes se procesan en conversaciones individuales.

SECCIÓN 02

Arquitectura del Sistema

El sistema está compuesto por **tres procesos independientes** que se ejecutan simultáneamente en el servidor Linux (Ubuntu). Esta arquitectura garantiza que si uno de los servicios presenta algún problema, los demás continúan funcionando sin interrupción.

Proceso	Puerto	Tecnología	Función principal
agente_pro.py	:8081	Python Flask	Núcleo central: WhatsApp, IA Claude, panel web, endpoints de sync
webhook_meli.py	:8080	Python Flask	Receptor exclusivo de notificaciones MercadoLibre (preguntas + órdenes)
bot-mckenna/server.js	:3000	Node.js	Bridge WhatsApp (whatsapp-web.js): enruta al agente :8081

2.1 Flujo de un Mensaje WhatsApp

Cuando un cliente escribe por WhatsApp, el recorrido del mensaje es el siguiente:

Paso	Componente	Acción
1	WhatsApp Business	Cliente envía mensaje al número de McKenna Group
2	bot-mckenna :3000	whatsapp-web.js recibe el mensaje y arma el JSON para el backend
3	POST /whatsapp :8081	Enruta según JID: grupos (contabilidad, pedidos web, ...) vs. chat directo
4	routes.py	Comandos de grupo o flujo de comprobante / modo humano / IA
5	Claude claude-sonnet-4-6	Procesa con tool use (Sheets, MeLi, Siigo, sync, etc.)
6	Herramientas	Consultas y acciones según la conversación
7	bot-mckenna	Devuelve la respuesta al cliente por WhatsApp

- Cuando el backend envía reportes al puente WhatsApp (Node :3000), si el servicio responde **503** (p. ej. "Sincronizando...") o hay un fallo de conexión breve, el sistema **reintenta** automáticamente antes de dar el envío por fallido. Así se reducen alertas perdidas tras un reinicio del bridge.

2.2 Modelo de IA

El cerebro del agente es **Claude claude-sonnet-4-6** de Anthropic, el modelo de IA más avanzado disponible con razonamiento extendido. Dispone de unas **32 herramientas registradas** que puede invocar autónomamente para responder consultas: catálogo en Google Sheets, MeLi y SIIGO, sync de facturas, precios, catálogo PDF, pipeline Facebook, crear facturas, consultar tarifas de envío, acceder a la memoria vectorial de casos aprendidos, entre otras.

■ **Aprendizaje continuo:** Cada vez que un operador responde manualmente una pregunta de preventa en MercadoLibre, la respuesta se guarda en una base de datos vectorial (ChromaDB). En el futuro, cuando llegue una pregunta similar, el agente usará ese caso como ejemplo para responder automáticamente. Esto mejora la precisión con el tiempo sin programación adicional.

2.3 Infraestructura y Seguridad

Capa de seguridad	Implementación	Protege contra
HTTPS público	Cloudflare Tunnel (sin puertos abiertos)	Exposición de IP, ataques de red
Autenticación API	Bearer Token en /sync/* y /chat	Acceso no autorizado a endpoints admin
API tienda web	Bearer WEB_API_KEY (donde aplique)	Actualizaciones de stock no autorizadas
OAuth MercadoLibre	Refresh token automático cada 6h	Token vencido = sin procesar órdenes
Rate Limiting	flask-limiter: 300 req/min global	Abuso, DDoS, scraping
Credenciales	Variables .env excluidas de git	Filtración de API keys en código
Backup nocturno	tar.gz cifrado en Google Drive a las 2 AM	Pérdida de datos por falla de disco

SECCIÓN 03

Módulos Funcionales

MOD-01 ■ Asistente WhatsApp con IA

Procesamiento de todos los mensajes de clientes por WhatsApp. El agente analiza el contexto completo (historial, perfil del cliente, modo IA/humano) y responde usando el catálogo oficial de SIIGO. Gestiona cotizaciones, consultas técnicas, disponibilidad de productos y coordina pagos. Tono ejecutivo colombiano ("veci").

MOD-02 ■ Preventa MercadoLibre Automática

Responde automáticamente preguntas de compradores en MeLi en menos de 30 segundos. Consulta la ficha técnica del producto en Google Sheets (columna I) y genera la respuesta con IA. Si no hay ficha técnica o falla la IA, delega al operador con una alerta en el grupo de WhatsApp.

MOD-03 ■ Gestión de Pagos y Comprobantes

Cuando un cliente envía una imagen, la clasifica como comprobante y alerta al grupo de contabilidad con los últimos 3 dígitos del número para identificación rápida. Comandos: ok 463 (confirma) / no 463 (rechaza). Monitor alerta si hay pagos sin confirmar más de 30 minutos.

MOD-04 ■ Sincronización Bidireccional de Stock

Cada venta en MeLi actualiza el stock en la página web (API) y viceversa. Principio: cada canal es fuente de verdad de su propio stock cuando vende. Función central: `sincronizar_stock_todas_las_plataformas` + `sincronizar_productos_pagina_web`. Sincronizaciones masivas: CLI o endpoints `/sync/*` con Bearer.

MOD-05 ■ SIIGO: Dos Flujos Independientes

FLUJO A — Registro de compra (proveedores especiales): procesa facturas electrónicas de proveedores de materias primas desde Gmail (XML UBL 2.1 DIAN), genera código McKenna, convierte unidades (LTR→mL, KGM→g), genera Excel de importación masiva y XML de compra con códigos internos para cargar en SIIGO con "Crear compra o gasto desde un XML o ZIP". Solo aplica a proveedores de materias primas inventariables configurados en la lista. | FLUJO B — Facturación de venta (clientes): crea factura electrónica SIIGO cuando se confirma un pago, cruza órdenes MeLi con el Pack ID, descarga PDF y lo adjunta en MeLi.

MOD-06 ■ Catálogo PDF Corporativo

Genera el catálogo de productos en PDF con diseño McKenna Group: lee el inventario de Google Sheets, descarga fotos reales de MeLi por `item_id`, genera tarjetas de producto con foto, SKU, precio tachado y precio con descuento. Envía el PDF al grupo de WhatsApp.

MOD-07 ■ Monitor de Alertas 24/7

Daemon que verifica continuamente el estado del sistema. Reinicia servicios caídos, alerta pagos pendientes, audita preguntas MeLi sin responder, refresca token OAuth, envía resumen diario a las 7 PM, reporte financiero semanal los lunes, e informe mensual el día 1 de cada mes.

MOD-08

■ Seguimiento Post-Venta

24 horas después de cada venta confirmada, envía automáticamente un mensaje de seguimiento al cliente por WhatsApp preguntando por su experiencia. Registra historial de compras por cliente en SQLite, distingue clientes nuevos vs. recurrentes y personaliza el saludo.

MOD-09

■ Despachos e Interrapidísimo

Genera guías de despacho internas con numeración automática. Consulta tarifas de Interrapidísimo por ciudad y peso/volumen. El producto más despachado aparece como "Producto Estrella" en el reporte financiero semanal.

MOD-10

■ Backup Nocturno en Google Drive

Cada noche a las 2 AM comprime bases de datos SQLite, archivos JSON de estado, embeddings ChromaDB y datos de training en un .tar.gz. Lo sube a Google Drive y guarda copia local. Limpia backups de más de 7 días. Notifica al grupo WhatsApp.

SECCIÓN 04

Guía de Comandos — Grupo de Contabilidad

El grupo de contabilidad en WhatsApp es el **panel de control operativo** del agente. Los mensajes enviados en este grupo son interceptados por el servidor antes de llegar a la IA, permitiendo que el equipo ejecute comandos directos sobre el sistema. Todos los comandos son **insensibles a mayúsculas/minúsculas**.

- Los comandos del grupo de contabilidad SOLO funcionan en el grupo de contabilidad registrado. Si se envían desde una conversación individual, el agente los tratará como mensajes normales de cliente.

4.1 Confirmación de Pagos

Comando	Ejemplo	Acción
ok <últimos 3 dígitos>	ok 463	Confirma el pago pendiente del cliente cuyo número termina en 463. El cliente recibe: "Veci, confirmamos su pago ■"
no <últimos 3 dígitos>	no 463	Rechaza el pago. El cliente recibe aviso de que hay un problema con la transacción.
ok confirmado	ok confirmado	Confirma si hay exactamente 1 pago pendiente. Si hay más de uno, el sistema pide especificar los dígitos.
ok confirmado	ok confirmado 573001234567@c.us	Confirma el pago del número completo especificado (formato legado).

- **¿Cómo identificar el código?** Cuando un cliente envía una imagen (comprobante de pago), el agente envía al grupo un mensaje como: "■ ALERTA DE PAGO — Cliente ...4567 — ■ Para CONFIRMAR: ok 567". Use siempre los últimos 3 dígitos que aparecen en esa alerta.

4.2 Respuestas Preventa MercadoLibre

Cuando el agente no puede responder automáticamente una pregunta de MeLi (sin ficha técnica o IA no disponible), envía una alerta al grupo con el ID de la pregunta. El operador responde usando estos comandos:

Comando	Ejemplo	Acción
resp <3 últimos dígitos del ID>:	resp 497: Hola, sí viene en polvo en presentación de 500g.	Responde la pregunta MeLi pendiente. El ID se muestra en la alerta del grupo. La respuesta queda guardada como caso de entrenamiento.
resp preventa :	resp preventa 13553987497: respuesta aquí	Formato completo con el ID entero (para mayor precisión).

4.3 Control IA / Modo Humano

Cuando un cliente requiere atención personalizada que la IA no puede brindar (negociaciones especiales, reclamos, devoluciones), el operador puede pausar la IA para ese número y atender manualmente:

Comando	Ejemplo	Acción
pausar	pausar 573001234567@c.us	Desactiva la IA para ese número. El cliente recibe: "En este momento te va a atender Jennifer García del área de ventas ■". Los mensajes del cliente se reenvían al grupo.
activar	activar 573001234567@c.us	Reactiva la IA para ese número. El cliente recibe saludo de bienvenida de vuelta.

4.4 Respuestas Directas y Aprobaciones

Comando	Ejemplo	Acción
resp :	resp 573001234567@c.us: Hola, su pedido ya fue despachado.	Envía un mensaje directo a un cliente desde el grupo sin que la IA intervenga.
hugo dale ok	hugo dale ok 1234567890	Aprueba y envía una respuesta de posventa que está pendiente de revisión humana.
ok	ok	Aprueba la siguiente factura de compra de proveedor en cola (importada desde Gmail).

4.5 Postventa MercadoLibre (grupo dedicado)

Los mensajes de compradores **después de la compra** en MeLi generan alertas en el grupo **GRUPO_POSTVENTA_WA**. La integración usa la API de mensajes en versión actual; si el comprador adjunta solo un PDF o imagen (p. ej. RUT) sin texto, la alerta indica los adjuntos y conviene abrir el hilo en MercadoLibre. Para responder desde WhatsApp usando el código que viene en la alerta:

Comando	Ejemplo	Acción
posventa :	posventa 3240: Hola, su pedido ya salió.	Envía la respuesta al pack de MeLi vinculado a ese código (estado en mensajes_posventa_pendientes.json).

- Si la IA generó un borrador de respuesta para un caso postventa, el mensaje de alerta pide aprobar con **hugo dale ok** y el ID de orden indicado. Si el reporte a WhatsApp falla tras una respuesta automática de preventa, revise el log del proceso :8080: queda registro para no perder trazabilidad.

SECCIÓN 05

Flujo de Atención al Cliente por WhatsApp

Desde el punto de vista del cliente, la conversación con Hugo García es fluida y natural. A continuación se describe cada etapa del proceso de atención completo, desde el primer contacto hasta la facturación y confirmación de despacho.

Paso 01 Primer Contacto / Saludo

Si el cliente saluda con "Hola", "Buenas tardes" u otro saludo sin hacer una pregunta específica, Hugo García responde EXACTAMENTE: "Hola Soy hugo Garcia de mckenna Group S.A.S, cuenteme en que le puedo servir veci!" — sin títulos largos ni información adicional no solicitada.

Paso 02 Consulta de Producto

El agente usa la herramienta buscar_producto_completo para consultar Google Sheets con el nombre del producto. Informa si está disponible o no, pero NO dice la cantidad exacta en stock. Solo si el cliente pregunta por una cantidad específica, confirma si esa cantidad está disponible.

Paso 03 Solicitud de Cotización

Si el cliente solicita una cotización, Hugo García recopila los datos paso a paso: (a) Nombre completo / razón social + NIT o cédula (b) Correo electrónico (c) Dirección de envío (d) Productos solicitados con cantidad. Luego genera una cotización preliminar local e informa al cliente que debe pagar y enviar el comprobante.

Paso 04 Envío del Comprobante de Pago

El cliente envía una imagen (comprobante). El agente la guarda en comprobantes/ con timestamp, alerta al grupo de contabilidad y responde al cliente: "Veci, recibí su comprobante. En un momento nuestro equipo de contabilidad lo verifica y le confirmamos."

Paso 05 Confirmación y Facturación

Cuando el operador confirma con "ok 463", el agente procede a: crear la factura electrónica en SIIGO ERP, descargar el PDF, subirlo al expediente de MeLi si fue una venta de MeLi, y enviar al cliente la confirmación con los datos de despacho.

Paso 06 Seguimiento Post-Venta

24 horas después, el agente envía automáticamente un mensaje de seguimiento al cliente preguntando por su experiencia y satisfacción con el producto recibido. Este paso ocurre sin intervención humana.

SECCIÓN 06

Preventa MercadoLibre

La preventa de MercadoLibre es el módulo más crítico en términos de velocidad de respuesta. Los compradores de MeLi esperan respuestas en minutos; el agente logra responder en menos de 30 segundos cuando tiene la información necesaria.

6.1 Árbol de Decisión

Condición	Acción del agente	Tiempo
Producto tiene ficha técnica en Google Sheets (col I) y Gemini responde OK	Genera y publica respuesta automática en MeLi	< 30 segundos
Producto tiene ficha técnica PERO la IA falla (timeout, error 503)	Envía alerta al grupo preventa MeLi (GRUPO_PREVENTA_WA) para respuesta manual	Inmediato
Producto NO tiene ficha técnica en Google Sheets	Envía alerta al mismo grupo preventa con la pregunta y nombre del producto sin ficha	Inmediato
Respuesta manual del operador (resp 497: ...)	Publica la respuesta en MeLi, guarda como caso de entrenamiento en ChromaDB	Al recibir el comando

- Si la respuesta ya quedó publicada en MeLi pero el aviso a WhatsApp falló tras varios reintentos, el servidor escribe un error en **consola / log** del proceso :8080. Revise el puente Node (:3000) y el JID del grupo preventa en **.env**.

6.2 Fichas Técnicas en Google Sheets

El sistema busca la ficha técnica del producto en la **columna I** de la hoja "BASE DE DATOS MCKENNA GROUP S.A.S" en Google Sheets. Si esa celda está vacía, el agente no puede responder automáticamente.

- **Acción recomendada:** Cada lunes a las 9 AM, el monitor envía automáticamente al grupo de contabilidad una lista de los productos que NO tienen ficha técnica (columna I vacía). Completar estas fichas aumenta directamente el porcentaje de respuestas automáticas y reduce la carga del equipo de ventas.

6.3 Aprendizaje Continuo (ChromaDB)

Cada respuesta manual procesada con el comando **resp <id>: <respuesta>** se guarda automáticamente como un caso de entrenamiento en dos lugares: • **app/training/casos_preventa.json**: archivo JSON con el historial de Q&A • **memoria_vectorial/**: base de datos ChromaDB con embeddings vectoriales Cuando llega una nueva pregunta similar, el agente recupera los casos más parecidos y los usa como ejemplos (few-shot learning) para generar una respuesta más precisa. El sistema mejora automáticamente a medida que el equipo responde preguntas.

SECCIÓN 07

Sincronización de Inventario

El inventario se mantiene sincronizado entre MercadoLibre y la **página web** (API REST). El principio fundamental es: **cada canal es fuente de verdad de su propio stock cuando vende**. No existe un "master de stock" externo que pueda generar conflictos.

Evento	Proceso automático	Resultado
Venta en MercadoLibre	MeLi autodecrementa → se lee SKU y available_quantity → sincronizar_stock_todas_las_plataformas hacia API web + MeLi	Web y MeLi alineados
Venta en la tienda web	La web decrementa su stock → el backend lee stock post-venta → actualizar_stock_meli en publicaciones por SKU	MeLi alineado con la web
Sincronización manual	CLI opción 3 (web), /sync/* o herramientas internas	Según el flujo ejecutado (reporte, auditoría SKUs, push a API)

7.1 Endpoints de Sincronización Manual

Los siguientes endpoints requieren el header **Authorization: Bearer <CHAT_API_TOKEN>**:

Endpoint	Método	Función
/sync/hoy	POST	Sincroniza facturas MeLi-SIIGO del último día
/sync/10dias	POST	Sincroniza facturas de los últimos 10 días
/sync/completo	POST	Sync completo de stock + reporte por WhatsApp
/sync/inteligente	POST	Cruza órdenes MeLi pendientes con facturas SIIGO
/sync/pack	POST	Sync de un Pack ID específico (body: {"pack_id": "xxx"})
/sync/fecha	POST	Sync por fecha específica (body: {"fecha": "YYYY-MM-DD"})
/sync/stock	POST	Genera reporte de stock y lo envía por WhatsApp
/sync/gmail	POST	Importa facturas de compra de proveedores desde Gmail

■ ■ **Nota sobre SIIGO:** SIIGO ERP se usa exclusivamente para facturación. El stock de SIIGO NO se sincroniza con MeLi ni con la web. La fuente de stock al vender son MeLi y la tienda web, cruzadas entre sí.

SECCIÓN 08

Facturación SIIGO y Documentos Fiscales

El agente gestiona el ciclo completo de facturación electrónica conforme a la normativa DIAN. Existen dos flujos: facturas de **venta** (hacia clientes) y facturas de **compra** (de proveedores, recibidas por Gmail).

8.1 Facturación de Venta (Ciclo Completo)

Paso	Descripción
1. Cotización preliminar	El agente crea un archivo JSON local en cotizaciones_preliminares/ con los datos del cliente y productos. No usa SIIGO aún.
2. Pago confirmado	Operador confirma con "ok 463". El agente ejecuta crear_factura_completa_siigo().
3. Factura en SIIGO	Se crea la factura electrónica oficial en SIIGO ERP con los datos de la cotización. SIIGO envía a la DIAN.
4. PDF descargado	El agente descarga el PDF de la factura de SIIGO y lo guarda en facturas_descargadas/.
5. Upload a MeLi	Si la venta fue por MercadoLibre, el PDF se adjunta automáticamente al expediente de la orden en MeLi como documento fiscal.
6. Notificación	Se envía al grupo de WhatsApp el resumen de la factura con el PDF adjunto y los datos de despacho.

8.2 Flujo A — Registro de Compra de Materias Primas (Gmail + XML DIAN)

¡**ATENCIÓN!** Este flujo es completamente independiente de la facturación de venta. Solo aplica a proveedores de **materias primas inventariables** (jabones, aceites, químicos, etc.). Las facturas de consumibles, gastos de envío o servicios se registran directamente en SIIGO como gasto/costo, sin pasar por este módulo.

- **Lista de proveedores especiales:** Configurar en `app/data/proveedores_especiales.json` los NITs de los proveedores de materias primas. Si la lista está vacía, el sistema procesa todos los proveedores (modo permisivo). Solo los proveedores listados como activos pasan por el proceso de codificación de inventario.

Paso	Descripción
1. Gmail OAuth	El agente se autentica en Gmail y busca correos con label "FACTURAS MCKG" que tengan archivos .zip adjuntos.
2. Filtro proveedor	Verifica si el NIT o nombre del proveedor está en <code>app/data/proveedores_especiales.json</code> . Si no está, la factura se omite con mensaje claro.
3. Extracción ZIP	Descomprime el archivo y extrae el XML DIAN (formato UBL 2.1 estándar colombiano).
4. Parseo XML	Lee: descripción del producto, cantidad, unidad DIAN (LTR, KGM, GLL, NAR, etc.), subtotal e IVA por línea.
5. Conversión unidades	Convierte a unidad mínima de inventario: LTR→mL (×1.000), KGM→g (×1.000), GLL→mL (×3.785), GRM→g (×1), etc.
6. Código McKenna	Genera código interno SIIGO: 3 letras × hasta 3 palabras clave (sin stopwords) + sufijo de unidad. Ejemplo: RICINOLTR → ACERICmL.

Paso	Descripción
7. Verificación SIIGO	Consulta API SIIGO para marcar productos ya existentes como DUPLICADO (se incluyen en el Excel para revisión pero no se sobrescriben).
8. Excel de importación	Genera archivo: "[Factura] registro productos.xlsx" con columnas: Tipo, Categoría, Código, Nombre, Inventariable, Unidad DIAN, Precio/unidad.
9. XML de compra	Genera archivo: "[Factura] codigos siigo.xml" con los códigos internos McKenna, cantidades convertidas, precios con IVA y protocolo de carga.
10. Notificación WA	Envía al grupo de contabilidad el resumen + Excel + XML adjuntos con el protocolo de pasos a seguir en SIIGO.

8.3 Protocolo de Carga en SIIGO

Una vez recibidos los archivos por WhatsApp, seguir estos pasos en SIIGO Nube:

Paso	Acción en SIIGO	Archivo
Paso A1	SIIGO → Inventario → Productos → ■ Importación	Excel adjunto
Paso A2	Seleccionar el archivo Excel en el Paso 2 del asistente	Excel adjunto
Paso A3	Verificar la vista previa: revisar los productos marcados como DUPLICADO antes de confirmar	—
Paso B1	SIIGO → Compras → Crear compra o gasto desde un XML o ZIP	XML adjunto
Paso B2	Cargar el archivo XML codigos siigo.xml	XML adjunto
Paso B3	Verificar que el TotalGeneral en SIIGO coincide con el total de la factura original del proveedor	—
Paso B4	Asentar el documento para registrar la compra en contabilidad	—

■ ■ **Pasos A y B son independientes:** El Paso A registra los productos en el catálogo de inventario. El Paso B registra la compra en la contabilidad. Ambos deben hacerse para que el inventario y la contabilidad queden correctamente actualizados en SIIGO. Si todos los productos son duplicados, el Paso A puede omitirse.

8.4 Tabla de Conversión de Unidades

Código DIAN	Nombre	Unidad mínima	Factor
LTR	Litro	mL	× 1.000
KGM	Kilogramo	g	× 1.000
GLL	Galón US	mL	× 3.785
GRM	Gramo	g	× 1
MLT	Mililitro	mL	× 1
LBR	Libra	g	× 453,59
NAR	Unidad (artículo)	Un	× 1

Código DIAN	Nombre	Unidad mínima	Factor
DZN	Docena	Un	× 12

SECCIÓN 09

Panel Web y Menú CLI

9.1 Panel Web de Control

Disponible en <http://<servidor>:8081/panel> (o vía Cloudflare Tunnel públicamente). El panel se actualiza automáticamente cada 60 segundos para servicios y 30 segundos para métricas.

Sección del panel	Información mostrada
KPIs del día	Mensajes WA atendidos, preguntas MeLi respondidas, pagos confirmados, órdenes sincronizadas
Estado de servicios	Badges ACTIVO/CAÍDO para agente-pro :8081, webhook-meli :8080, whatsapp :3000
Cola de preventa MeLi	Lista de preguntas pendientes con botón "Responder" que ejecuta resp directamente
Integraciones	Estado de integraciones (MeLi, SIIGO, web, Google Sheets, Gmail, Drive, WA, Cloudflare)
Tasa de automatización	Porcentaje de preguntas MeLi respondidas automáticamente vs. total
Log de actividad	Últimas acciones ejecutadas por el agente en tiempo real

9.2 Menú CLI Interactivo

El menú CLI se inicia automáticamente cuando se ejecuta el servidor. Es accesible desde la terminal del servidor donde corre el agente. Permite ejecutar operaciones manuales sin necesidad de acceder a la API:

Opción	Función
1 — ■ CHAT	Conversación directa con Hugo García desde la terminal (útil para pruebas y depuración)
2 — ■ FACTURAS	Menú con opciones de sincronización MeLi ↔ SIIGO (Inteligente, Hoy, Días, Pack, Fecha)
3 — ■ STOCK	Menú de inventario: Reporte completo, Verificar SKUs, Sincronizar Web (API tienda)
4 — ■ CONSULTA	Busca información detallada de un producto en el catálogo de Google Sheets
5 — ■ APRENDIZAJE	Fuerza la extracción de Q&A; recientes en MeLi para entrenamiento en ChromaDB
6 — ■ COMPRAS	Importa facturas electrónicas de proveedores desde Gmail (XML DIAN) y procesa SIIGO
7 — ■ CIENCIA	Genera contenido científico automatizado y lo publica en WordPress (Knowledge Agent)
8 — ■ SALIR	Cierra el menú CLI (el servidor Flask web sigue corriendo en segundo plano)

SECCIÓN 10

Monitor de Alertas Automáticas

El monitor es un daemon (proceso en segundo plano) que corre continuamente dentro del servidor. Verifica el estado del sistema cada minuto y ejecuta tareas programadas automáticamente. Todos los reportes y alertas se envían al grupo de contabilidad en WhatsApp.

10.1 Alertas y Tareas Programadas

Frecuencia	Tarea	Destino
Cada 5 minutos	Verifica que agente-pro :8081, webhook-meli :8080 y whatsapp :3000 respondan. Si alguno falla, intenta reiniciarlo con systemctl y alerta al grupo.	Grupo WA
Cada 15 minutos	Revisa comprobantes de pago sin confirmar hace más de 30 minutos y alerta al grupo.	Grupo WA
Cada 30 minutos	Cuenta preguntas de MeLi sin responder. Si hay más de 3, alerta al grupo con la última pregunta.	Grupo WA
Cada 6 horas	Verifica vencimiento del token OAuth de MercadoLibre. Si vence en menos de 60 min, lo refresca automáticamente.	Log interno
08:00 AM diario	Ejecuta sincronización de stock diario entre plataformas y envía reporte al grupo.	Grupo WA
07:00 PM diario	Envía resumen del día: servicios activos, mensajes WA, preguntas MeLi, pagos, órdenes.	Grupo WA
02:00 AM diario	Backup nocturno: comprime datos críticos en .tar.gz y sube a Google Drive.	Grupo WA + Drive
Lunes 07:00 AM	Reporte financiero semanal HTML por correo: total facturado SIIGO, producto estrella, clientes nuevos vs. recurrentes.	cynthua0418@gmail.com
Lunes 09:00 AM	Lista de productos sin ficha técnica (columna I vacía en Sheets) para que el equipo las complete.	Grupo WA
Día 1 de cada mes 08:00 AM	Informe forense mensual completo del agente: métricas, estado, recomendaciones, optimizaciones.	cynthua0418@gmail.com

- **Diagnóstico de errores:** Si el agente responde con un mensaje de error técnico, el error completo se registra en **log_errores_ia.txt** en la raíz del proyecto. Para verlo en tiempo real: **tail -f /home/mckg/mi-agente/log_errores_ia.txt**

SECCIÓN 11

Glosario Técnico y Preguntas Frecuentes

11.1 Glosario de Términos

Término	Definición
bot-mckenna	Servicio Node.js (whatsapp-web.js) en :3000 que enlaza WhatsApp con POST /whatsapp :8081
Pack ID	Identificador único de un paquete de órdenes en MercadoLibre. Sirve para adjuntar facturas fiscales
Ficha técnica	Texto en la columna I de Google Sheets con descripción técnica detallada del producto para respuestas de preventa
ChromaDB	Base de datos vectorial que almacena embeddings de preguntas y respuestas para búsqueda semántica
Few-shot learning	Técnica de IA donde se le muestran al modelo ejemplos de respuestas correctas para que aprenda el patrón
WEB_API_KEY	Bearer para la API de stock/precios de la tienda web cuando el backend empuja catálogo
Bearer Token	Cadena de texto secreto que se incluye en el header de peticiones HTTP para autenticar acceso a la API
OAuth 2.0	Protocolo estándar de autorización usado por MercadoLibre y Gmail para tokens de acceso
UBL 2.1	Formato XML estándar colombiano (DIAN) para facturas electrónicas de proveedores
Daemon	Proceso que corre en segundo plano continuamente sin interfaz de usuario (ej: el monitor de alertas)
SKU	Stock Keeping Unit — código único que identifica un producto en todas las plataformas
Cloudflare Tunnel	Servicio que expone el servidor local al internet de forma segura sin abrir puertos ni exponer la IP
Proveedor especial	Proveedor de materias primas inventariables registrado en app/data/proveedores_especiales.json. Solo estos pasan por el flujo de codificación McKenna.
Código McKenna	Código interno de producto SIIGO generado automáticamente: 3 letras iniciales × 3 palabras clave del nombre + sufijo de unidad (mL, g, Un). Ej: ACERICmL.
Flujo A (compra)	Proceso de registro de compra de materias primas en SIIGO: Gmail → ZIP → XML DIAN → codificar → importar inventario + registrar compra.
Flujo B (venta)	Proceso de facturación de venta a clientes en SIIGO: pago confirmado → crear factura → PDF → subir a MeLi. Completamente independiente del Flujo A.
Unidad mínima	Unidad de medida base para el inventario en SIIGO: mL para líquidos, g para sólidos, Un para artículos. Las cantidades del proveedor se convierten a esta unidad.

11.2 Preguntas Frecuentes (FAQ)

■ ¿Por qué el agente respondió "Veci, tuve un problema técnico momentáneo"?

Ocurrió un error inesperado. Ver log_errores_ia.txt para el detalle. Causas comunes: credencial de Google Sheets vencida, servicio SIIGO no disponible, o error de red con la API de Claude. El cliente puede repetir el mensaje; el historial de conversación se preserva entre intentos.

■ ¿El agente puede responder preguntas que no sean de productos?

Sí, con limitaciones. El agente puede responder preguntas generales sobre McKenna Group, procesos de pago, tiempos de envío (usando tarifas de Interrapidísimo), y datos básicos de facturación. Para información muy específica que no está en sus herramientas, escalará al grupo de contabilidad.

■ ¿Cómo sé si una ficha técnica está bien configurada?

El agente la leerá automáticamente de la columna I de Google Sheets. Si el agente responde la pregunta de MeLi automáticamente, la ficha funciona. Si envía alerta al grupo, la ficha está vacía o el producto no está en el Sheet.

■ ¿Qué pasa si el token de MercadoLibre vence?

El monitor verifica el token cada 6 horas. Si queda menos de 60 minutos para vencer, lo refresca automáticamente con el refresh_token guardado en credenciales_meli.json. Si el refresh también falla, el agente alerta al grupo y las sincronizaciones pausan hasta renovar.

■ ¿Cómo agrego un nuevo producto al catálogo?

Agregar el producto en Google Sheets en la pestaña "BASE DE DATOS MCKENNA GROUP S.A.S" con: col A (ID de publicación MeLi), col D (nombre oficial SIIGO), col I (ficha técnica), y las columnas de precio y stock. El agente lo verá inmediatamente en la próxima consulta.

■ ¿Puedo hacer que el agente NO responda a un cliente específico?

Sí. En el grupo de contabilidad escribe: **pausar 573XXXXXXXXX@c.us**. El agente avisará al cliente que lo atenderá Jennifer García. Para reactivar: **activar 573XXXXXXXXX@c.us**

■ ¿Dónde se guardan los comprobantes de pago?

En la carpeta comprobantes/ del proyecto, con nombre {número}_{timestamp}.jpeg. Se guardan indefinidamente como registro. Limpiar periódicamente los más antiguos para no acumular espacio en disco.

■ ¿Cuál es la diferencia entre el Flujo A y el Flujo B de SIIGO?

Son dos procesos completamente independientes y no deben confundirse: • Flujo A (compra): cuando McKenna COMPRA materias primas a un proveedor. El agente procesa la factura electrónica del proveedor (XML DIAN) desde Gmail, genera códigos internos y produce el Excel + XML para registrar en SIIGO la entrada de inventario. • Flujo B (venta): cuando McKenna VENDE a un cliente. El agente crea la factura electrónica de venta en SIIGO y la adjunta a la orden en MercadoLibre. Un proveedor que vende consumibles o servicios (empaquete, envíos, etc.) se registra directamente en SIIGO como gasto, sin pasar por el Flujo A.

■ ¿Cómo agrego un proveedor de materias primas para que el agente lo procese automáticamente?

Editar el archivo `app/data/proveedores_especiales.json` y agregar una entrada con el NIT del proveedor, su nombre, `activo: true` y una nota descriptiva. Ejemplo: `{ "nit": "900123456", "nombre": "PROVEEDOR S.A.S.", "activo": true, "nota": "Aceites y alcoholes" }` Si la lista está vacía, el agente procesa todos los proveedores (modo permisivo). Esto es útil durante la configuración inicial.

■ ¿Cómo se genera el código McKenna de un producto?

El código se construye automáticamente a partir del nombre del producto en la factura del proveedor: 1. Se normalizan las palabras (sin tildes, mayúsculas) 2. Se eliminan stopwords (de, del, la, el, para, con, etc.) 3. Se toman las 3 primeras letras de hasta 3 palabras clave 4. Se agrega el sufijo de unidad mínima (mL, g, Un) Ejemplo: "ACEITE DE RICINO" en litros → ACE + RIC = ACERICmL El mismo código se usa en el Excel de importación y en el XML de compra.

SECCIÓN 12

Generación de Contenido Científico y Web

El agente cuenta con un sistema completo de investigación científica automatizada y producción de contenido multimedia. Este sistema opera en dos capas: (1) búsqueda y síntesis de literatura científica usando bases de datos públicas, y (2) producción audiovisual para redes sociales. Todo el contenido generado está enriquecido con referencias reales de PubMed y ArXiv, lo que otorga credibilidad técnica a la marca McKenna Group.

12.1 Knowledge Agent — Investigación Científica Automática

El módulo `app/tools/knowledge_agent.py` es el motor central de investigación. Consulta bases de datos científicas internacionales, extrae los hallazgos más relevantes para el sector farmacéutico/cosmético y genera contenido estructurado listo para publicar.

Fuente	URL / API	Credenciales	Tipo de contenido
PubMed (NCBI)	<code>eutils.ncbi.nlm.nih.gov</code>	Sin clave (gratuita)	Artículos clínicos, abstracts, estudios farmacéuticos
ArXiv	<code>export.arxiv.org/api/query</code>	Sin clave (gratuita)	Preprints de nanomateriales y tendencias emergentes
Scrapling / Web	Cualquier URL pública	Sin clave	Texto de artículos, fichas técnicas externas

Flujo completo del Knowledge Agent:

→ 1. `buscar_pubmed(termino)` Consulta NCBI E-utilities. Usa filtros MeSH: `cosmetic[MeSH]` OR `pharmaceutical[MeSH]`. Si no retorna resultados, reintenta sin filtros. Extrae: PMID, título, abstract, autores, año, URL.

→	2. buscar_arxiv(termino) Consulta ArXiv API Atom. Parsea XML buscando , , . Retorna hasta 3 preprints relevantes.
→	3. scrape_url(url) Si se provee una URL externa, usa la librería scrapling para extraer texto. Fallback: requests + regex sobre etiquetas y . Límite: 4 000 caracteres.
→	4. Gemini enriquece Los abstracts y textos extraídos se pasan a Gemini como contexto. El modelo genera contenido estructurado (post, ficha, receta, manual de uso) integrando las referencias.
→	5. Almacena en ChromaDB El contenido generado se vectoriza y almacena en memoria_vectorial/ para que el agente pueda usarlo en respuestas de preventa MeLi.
→	6. Publica en WordPress Llama a la REST API de WordPress con autenticación Basic (Base64 de WP_USER:WP_APP_PASSWORD). El post queda publicado en mckennagroup.co.

12.2 Pipeline Multimedia para Facebook

El script **pipeline_contenido_facebook.py** conecta seis APIs en secuencia para producir un video de 20-25 segundos con voz, imagen generada por IA y paleta de marca McKenna, listo para publicar en la página de Facebook.

Pas o	Herramienta	Qué hace	Variable .env
1. C opy	Gemini 2.5-Flash	Genera título, narración (25-40 palabras), prompts de imagen y video, caption y hashtags	GOOGLE_API_KEY
2. I mag en	Ideogram AI	Genera fondo fotorrealista SIN texto (1200×630 px). Retorna bytes + URL pública	IDEOGRAM_API_KEY
3. C omp osici ón	PIL (Pillow)	Superpone título, datos técnicos, logo McKenna y paleta de marca (#143D36, #2E8B7A, #FFC83C) sobre el fondo	—
4. Voz	ElevenLabs TTS	Narración en español colombiano. Voz Liam, model eleven_multilingual_v2	ELEVENLABS_API_KEY
5. Vi deo IA	fal.ai / Kling v1.6	Video cinematográfico 10s×2 escenas desde imagen + prompts de movimiento. Fallback: Ken Burns con ffmpeg	FAL_KEY
6. M ezcl a	ffmpeg	Une video + MP3. Codec H.264, bitrate 5 000k, resampling de audio	—
7. P ublic ació n	Facebook Graph API v19	Sube el video con caption + hashtags a la página McKenna	FB_PAGE_TOKEN, FB_PAGE_ID

- Si fal.ai no tiene saldo, generar_video_ken_burns() genera el video localmente con ffmpeg aplicando un efecto de zoom cinematográfico (Ken Burns) sobre la imagen estática. No requiere conexión externa.

Comandos de uso desde terminal:

Comando	Descripción
python3 pipeline_contenido_facebook.py --tipo ficha --slug acido-ascorbico	Pipeline completo para un ingrediente específico
python3 pipeline_contenido_facebook.py --tipo receta --slug serum-vitamina-c	Pipeline completo para una receta
python3 pipeline_contenido_facebook.py --tipo comparativa	Comparativa de dos activos
python3 pipeline_contenido_facebook.py --tipo tip	Consejo de formulación
python3 pipeline_contenido_facebook.py --auto	Elige automáticamente el mejor contenido disponible
python3 generar_infografias_facebook.py --tipo receta --n 3	Genera 3 infografías estáticas (sin video)
python3 sincronizar_facebook.py	Borra todos los posts y republica la página completa

12.3 Scripts de Generación Masiva de Contenido Web

Estos scripts generan en lote el contenido técnico y científico que alimenta el sitio web mckennagroup.co. Cada script escribe un archivo JSON que el frontend consume directamente.

Script	Qué genera	Volumen	Output
generar_guias_masivas.py	62 guías de ingredientes farmacéuticos y cosméticos. Cada guía tiene 7 secciones HTML: descripción, concentraciones (tabla), compatibilidad, incorporación, almacenamiento, normativa INVIMA y FAQ. Integra referencias de PubMed.	62 ingredientes	PAGINA_WEB/site/data/guias.json (606 KB)
generar_posts_masivos.py	20+ posts comparativos de blog. Estructura: intro, hallazgos contrastados de estudios, gráficas SVG/CSS inline, bibliografía numerada. Usa PubMed con filtros MeSH.	20+ posts	PAGINA_WEB/site/data/posts.json (218 KB)
generar_recetas_masivas.py	40+ recetas de formulación en 4 categorías: cosmética (sérum, cremas), nutrición, perfumería y hogar. Cada receta incluye ingredientes con cantidades, modo de preparación y precauciones.	40+ recetas	PAGINA_WEB/site/data/recetas.json (77 KB)

12.4 Variables de Entorno Requeridas

Variable	Módulo que la usa	Descripción
GOOGLE_API_KEY	knowledge_agent, pipeline_facebook, guías, posts, recetas	Clave de Google GenAI para Gemini 2.5
IDEOGRAM_API_KEY	pipeline_contenido_facebook.py	Clave de Ideogram para generación de imágenes

Variable	Módulo que la usa	Descripción
ELEVENLABS_API_KEY	pipeline_contenido_facebook.py	Clave de ElevenLabs para síntesis de voz TTS
FAL_KEY	pipeline_contenido_facebook.py	Clave de fal.ai para generación de video con Kling v1.6
FB_PAGE_TOKEN	pipeline_facebook, sincronizar_facebook, infografías	Token de acceso a la página de Facebook (Graph API)
FB_PAGE_ID	pipeline_facebook, sincronizar_facebook, infografías	ID numérico de la página de Facebook de McKenna Group
WP_USER	knowledge_agent.py	Usuario de WordPress con permisos de editor
WP_APP_PASSWORD	knowledge_agent.py	Application Password de WordPress (Usuarios → Seguridad)
WC_URL	knowledge_agent.py (como WP_URL)	URL base del sitio: https://mckennagroup.co

■ ■ El pipeline de Facebook requiere que ffmpeg esté instalado en el sistema (sudo apt install ffmpeg). fal.ai cobra por segundo de video generado; si el saldo es cero, el sistema usa automáticamente el fallback local de Ken Burns.

SECCIÓN 13

Nuevas Skills (Herramientas Autónomas)

El agente "Hugo García" ha evolucionado de ejecutar scripts manualmente a poseer **Skills nativos**. Múltiples scripts aislados han sido refactorizados y encapsulados en funciones estructuradas dentro de la carpeta **app/tools/**. El modelo de IA (Claude/Gemini) ahora conoce estas herramientas y decide cuándo ejecutarlas autónomamente basándose en la conversación con el usuario.

13.1 Arquitectura e Interacción

Cada Skill está documentada con Type Hints y Docstrings. En **app/core.py**, la función **_fn_to_tool_schema()** traduce automáticamente la función de Python a un esquema JSON compatible con el formato de herramientas de la IA (Tool Calling). Cuando le pides al agente realizar una tarea, este analiza sus "herramientas registradas", pide permiso para ejecutarlas con los argumentos deducidos, y el sistema ejecuta la función en Python, devolviendo el resultado (éxito o error) al agente para que te responda de forma natural.

13.2 Ejemplos de Skills

Nombre de Skill	Archivo	Descripción	Ejemplo de uso en el chat
sincronizar_precios_meli_sheets	sincronizar_precios.py	Precios MeLi → Google Sheets (y caché web si aplica).	"Hugo, actualiza los precios en el Sheets usando los de MeLi"
generar_catalogo_pdf	generar_catalogo.py	PDF corporativo con fotos MeLi; opcional envío WA.	"Genera el catálogo PDF corporativo de este mes"
sincronizar_inteligente / sincronizar_manual_por_id	app/sync.py	Facturas Siigo ↔ documentos fiscales MeLi.	"Sincroniza facturas con MeLi"
publicar_contenido_redes_sociales_ia	pipeline_contenido_facebook.py	Pipeline multimedia y publicación en Facebook.	"Haz un nuevo post para Facebook"
generar_guias_masivas_web	generar_guias_masivas.py	Guías técnicas en JSON para el sitio.	"Genera las guías masivas que faltan en la web"
sincronizar_productos_pagina_web	sincronizar_productos_pagina_web.py	Empuja stock/precios a la API de la tienda (CLI/sync; ver registro en core.py).	"Sincroniza el catálogo con la página web"
auditar_scripts	script_audit.py	py_compile de scripts del manifiesto; sin ejecutar main.	"Audita los scripts del manifiesto"

■ **Registro de Skills:** Para agregar una nueva, simplemente se debe crear la función en **app/tools/** y agregarla a la lista **todas_las_herramientas** dentro de la función **configurar_ia()** en **app/core.py**.

SECCIÓN 14

Observabilidad, seguridad de herramientas, auditoría y backup

Esta sección resume mejoras operativas del agente: trazabilidad de peticiones, límites a herramientas que modifican código, verificación diaria de scripts, backup nocturno con aviso por WhatsApp y sincronización opcional con GitHub.

14.1 Observabilidad (request_id y logs JSON)

Cada petición HTTP en **agente_pro** (8081) y **webhook_meli** (8080) recibe un **request_id** (UUID o cabecera **X-Request-ID**). Se propaga a los hilos en segundo plano (MeLi, WhatsApp) para correlacionar logs. Con **AGENTE_LOG_JSON=1** se imprimen eventos en una línea JSON (stderr). El bucle de herramientas de Claude registra **tool_ok**, **tool_error** e **ia_turn_start**. **GET /status** devuelve el **request_id** actual.

14.2 Herramientas de archivos (parche / ejecutar script)

Variable	Efecto
AGENTE_RESTRICT_FILE_TOOLS=1	Restringe parchear_funcion, crear_nuevo_script y ejecutar_script_python a rutas bajo prefijos del repo.
FLASK_ENV=production	Activa la misma restricción sin variable adicional.
AGENTE_FILE_TOOL_PREFIXES	Lista separada por comas (por defecto: scripts/,app/tools/,tests/).

14.3 Auditoría de scripts y cron

El manifiesto **app/data/scripts_manifest.json** lista .py críticos. La herramienta del agente **auditar_scripts** (y **ejecutar_auditoria_dict** en código) ejecuta **py_compile** sin correr el main. El script **scripts/auditar_scripts_cron.py** se puede programar con **scripts/instalar_cron_mckenna.sh** (7:15 diario; log en **log_cron.txt**). Si hay fallos, envía WhatsApp al grupo de alertas (salvo **AGENTE_AUDITORIA_SKIP_WA=1**).

14.4 Backup nocturno, GitHub y grupo de alertas

A las **2:00** el daemon en **app/tools/backup_drive.py** genera un .tar.gz en **backups_drive/** (datos JSON, training, ChromaDB, SQLite) y opcionalmente sube a Google Drive. Tras el backup intenta **git add -A**, **commit** y **git push origin <rama>** si hay cambios (desactivar con **AGENTE_NIGHTLY_GIT_PUSH=0**). La carpeta **backups_drive/** no se versiona en git. Los avisos de backup y el resultado del push van a **GRUPO_ALERTAS_SISTEMAS_WA** (función **jid_grupo_alertas_sistemas_wa()** en **app/utls.py**).

- Manual actualizado (v3.4). Regenerar PDF: **python3 scripts/generar_manual.py** · enviar por correo: **--enviar**.

SECCIÓN 15

Metodología Agentica McKenna

McKenna adopta una metodología agentica inspirada en el paso de "vibe coding" a desarrollo profesional con IA: el operador actúa como director de orquesta, un agente orquestador conserva el contexto limpio, y subagentes especializados exploran, implementan, verifican y revisan cambios sin cargar todo el proyecto en cada turno.

15.1 Ecosistema Gentleman integrado

Repositorio	Rol	Uso recomendado en McKenna
gentle-ai	Configurador central: SDD, skills, MCP, Engram, persona y subagentes.	Adopción objetivo para superagentes; instalar primero en dev.
engram	Memoria persistente agent-agnostic con SQLite + FTS5, CLI, HTTP API, MCP, TUI y sync.	Candidato para memoria orgánica/sync entre agentes.
agent-teams-lite	Orquestador + subagentes + SDD en Markdown; proyecto archivado.	Referencia histórica; preferir gentle-ai.
Gentleman-Skills	Catálogo de skills curadas/comunitarias.	Importar selectivamente React 19, TypeScript, pytest, Playwright y github-pr.
gentleman-guardian-angel	Revisión AI provider-agnostic para commits/PRs.	Usar como review no bloqueante antes de activar hooks.
Gentleman.Dots	Entorno dev: editor, shells, terminales, tmux/zellij.	Opcional para máquinas dev; la capa AI vive en gentle-ai.

15.2 Flujo operativo y delegación automática

Paso	Acción	Archivo de referencia
1	Identificar módulo principal y secundarios	docs/agentica/INDEX.md
2	Consultar memoria de bugs, decisiones e invariantes	docs/agentica/MEMORY.md
3	Cargar skill/ficha mínima según intención	docs/agentica/SKILLS.md
4	Delegar exploración readonly si el cambio cruza capas	docs/agentica/ORCHESTRATION.md
5	Planear, implementar acotado y verificar	docs/agentica/CHECKLIST.md
6	Actualizar contratos y guardar aprendizaje reusable	docs/agentica/CONTRACTS.md / DECISIONS.md

Petición natural	Skill/subagente
'revisa preguntas de MeLi / messages / orders_v2'	webhook-meli + Explore/Verify
'arregla WhatsApp / resp / posventa / comprobante'	whatsapp-routes + Explore
'agrega herramienta a Hugo / cambia prompt Claude'	core-tools + Review

Petición natural	Skill/subagente
'stock / facturas / Siigo / sync web'	sync-stock + Explore/Verify
'panel React / dashboard / Vite'	desktop-panel + Verify
'systemd / puerto / deploy / nohup'	ops-systemd + Explore
'review PR / pre-commit / estándares'	guardian-review

15.3 Componentes creados

Componente	Propósito
docs/agentic/INDEX.md	Mapa rápido de lectura por tipo de cambio para ahorrar tokens.
docs/agentic/ORCHESTRATION.md	Roles: orquestador, explore, implement, verify y review.
docs/agentic/MEMORY.md	Reglas para memoria local tipo Engram usando SQLite/Chroma/debug-memory existente.
docs/agentic/SKILLS.md	Registro de skills lazy por módulo y triggers de uso.
docs/agentic/ECOSYSTEM.md	Mapa de gentle-ai, Engram, ATL, Gentleman-Skills, GGA y Gentleman.Dots.
docs/agentic/modules/*.md	Fichas cortas por webhook MeLi, WhatsApp, core tools, sync, desktop, ops y QA.
docs/agentic/CONTRACTS.md	Contratos críticos de /api, /chat, /whatsapp y /notifications.
docs/agentic/DECISIONS.md	Memoria versionada de decisiones técnicas y agenticas.
docs/agentic/learned_context.md	Resumen portable para compartir contexto con otros devs/agentes.

15.4 Validación automática agregada

Se restauró y amplió `tests/test_smoke.py` para validar auditoría de scripts, guards de herramientas de archivo, contratos puros de despacho MeLi, rutas Flask críticas y existencia de la documentación agentic. También se agregó `.github/workflows/backend-qa.yml` para ejecutar smoke tests y auditoría cuando cambie backend, scripts, tests, requirements o docs agentic.

Validación	Comando
Smoke backend	<code>venv/bin/python -m pytest tests/test_smoke.py</code>
Auditoría scripts	<code>AGENTE_AUDITORIA_SKIP_WA=1 AGENTE_AUDITORIA_CRON_QUIET=1 venv/bin/python scripts/auditar_scripts_cron.py</code>
QA panel React	<code>cd desktop && npm run qa:full</code>

- **Regla de memoria:** guardar solo conocimiento reutilizable: invariantes, bugs con causa raíz, contratos entre servicios, comandos de validación y decisiones con tradeoff. Engram/sync nunca debe guardar secretos, tokens, .env ni credenciales. No guardar ruido de cada edición.

SECCIÓN 16

Refactorización Agéntica 2026-05-24: AgentRun + Tricap Memory

El 2026-05-24 se realizó la mayor refactorización técnica del agente desde su creación. El bucle monolítico de tool-use que vivía directamente en `app/core.py` (unas 325 líneas) fue descompuesto en un paquete dedicado: `app/agent/`. Al mismo tiempo se creó un sistema de memoria de tres capas llamado **Tricap Memory** en `app/memory/`. El resultado: 96 de 96 tests en verde, reintentos automáticos ante fallos del proveedor LLM, checkpoints por iteración y un router multi-proveedor con fallback automático.

■ **Analogía:** Antes el agente era como un chef que cocinaba, servía, anotaba los pedidos y atendía la caja al mismo tiempo. Ahora hay un jefe de sala (**AgentRun**) que coordina, un proveedor de ingredientes intercambiable (**LLMRouter**), un despachador de platos (**ToolDispatcher**) y una libreta de pedidos con historial (**Tricap Memory**). Cada rol tiene responsabilidad única y puede cambiarse sin afectar al resto.

16.1 AgentRun — La Máquina de Estados del Turno

Archivo: `app/agent/run.py`. Es el director de cada turno de conversación. No conoce qué modelo ejecuta ni cómo se llaman las tools; solo orquesta el ciclo.

Concepto	Valor	Qué significa
MAX_TOOL_ITERS	20	Máximo de iteraciones de tool-use por intento antes de abortar con mensaje legible
MAX_REINTENTOS	3	Intentos con proveedores distintos si uno falla o agota sus peticiones
Contexto episódico	En reintentos	Si ya hubo tools fallidas, las inyecta en el system prompt del reintento para que el modelo no repita el mismo error
Checkpoint	Después de cada tool	Guarda en SQLite el estado completo de mensajes antes de continuar, para poder auditar o recuperar
Cleanup	Siempre	Borra los checkpoints del turno al terminar (éxito, límite o error total)

Paso	Acción
1. Inicio de turno	AgentRun recibe pregunta, mensajes de historial, adjuntos y mapa de tools
2. Bucle de reintentos (máx 3)	En cada intento, corre hasta 20 iteraciones de tool-use con el proveedor actual
3. LLM call	Llama a <code>router.complete(messages, tools, system)</code> . El router decide qué proveedor usar
4. ¿Necesita tools?	Si sí: llama a <code>ToolDispatcher</code> , guarda checkpoint, añade resultado a mensajes y vuelve al paso 3
5. ¿Fin de turno?	Si no hay más tools: hace cleanup, retorna <code>AgentResult(text, messages, provider, run_id, iterations)</code>
6. Límite de iteraciones	Si llega a 20 sin terminar: retorna mensaje de error legible para el usuario
7. Todos los proveedores fallan	<code>AllProvidersExhausted</code> : retorna mensaje de mantenimiento/reintento

16.2 LLMRouter — Multi-Proveedor con Fallback Automático

Archivo: `app/agent/llm_router.py`. Abstrae el acceso a modelos de IA con una interfaz uniforme. AgentRun nunca habla directamente con Anthropic, Google o Ollama; siempre usa el router, que decide cuál proveedor intentar y cuándo escalar.

Proveedor	Tool-use	Modelo típico	Cuándo se usa
ClaudeProvider	Completo	claude-sonnet-4-6	Primario para WhatsApp y /chat. Requiere ANTHROPIC_API_KEY.
GeminiProvider	No (solo texto)	gemini-2.5-pro	Canales que configuren Gemini o fallback sin tools. Requiere GOOGLE_API_KEY.
OllamaProvider	No (solo texto)	gemma4:latest	Último recurso local. Requiere ollama en PATH y modelo descargado.

■■ Regla crítica: Si `ANTHROPIC_API_KEY` no está en `.env`, `cliente_ia = None` y `core.py` devuelve "mantenimiento" antes de llegar al router. Sin esta clave no hay tool-use posible; GeminiProvider y OllamaProvider rechazan tools. El bot responderá "estamos en mantenimiento" hasta que se configure la clave.

Situación	Comportamiento del router
Proveedor primario falla 3 veces	Escala al siguiente en la cadena: Claude → Gemini → Ollama
Fallback no soporta tools y se necesitan tools	Salta ese proveedor, pasa al siguiente
Todos los proveedores fallan	Lanza AllProvidersExhausted → mensaje de mantenimiento

16.3 ToolDispatcher — Ejecución con Registro Episódico

Archivo: `app/agent/tool_dispatcher.py`. Recibe la lista de tool calls que el LLM quiere ejecutar y las despacha, manejando errores, límites de tamaño y registro de episodios.

Propiedad	Valor / Comportamiento
<code>_MAX_RESULT_CHARS</code>	8 192 caracteres — si el resultado de una tool es más largo, se trunca antes de devolverlo al LLM
Registro episódico	Cada tool ejecutada se registra en memoria episódica (JSONL por <code>run_id</code>) con resultado y timestamp
Error en tool	Lanza compresión y almacenamiento en hilo paralelo (no bloquea el turno). El error se devuelve como texto al LLM
<code>apply_web_overrides</code>	Para el canal <code>web_chat</code> redirige <code>buscar_producto_completo</code> → <code>buscar_productos_combo_siigo</code>

16.4 CheckpointStore — Persistencia por Iteración

Archivo: `app/agent/checkpoint_store.py`. Guarda el estado completo de mensajes en `app/data/agent_checkpoints.sqlite3` después de cada iteración de tools. Si el proceso se mata a mitad de turno, los checkpoints quedan en la base de datos para auditoría. Se limpian automáticamente: al terminar el turno (`delete_run`) y los de más de 24 horas (`purge_old`).

Función	Cuándo se llama
save(checkpoint)	Después de cada iteración de tools exitosa, antes de la siguiente LLM call
load_latest(run_id)	Para auditoría o recuperación manual (no se usa en el flujo normal)
delete_run(run_id)	Al finalizar el turno (éxito, límite o AllProvidersExhausted)
purge_old(24h)	Al inicio de cada turno nuevo, limpia checkpoints huérfanos de runs anteriores

16.5 Tricap Memory — Sistema de Memoria de Tres Capas

Carpeta: **app/memory/**. El agente tiene tres tipos de memoria que trabajan en paralelo durante cada turno, más un compresor que reduce el consumo de contexto.

Capa	Archivo	Qué almacena	Cuándo se usa
Working (trabajo)	working.py	Contexto activo del turno: historial de mensajes en memoria RAM. Tiene un límite de tokens; llama <code>evict_if_needed()</code> antes de cada LLM call para recortar mensajes viejos si el contexto está lleno.	Antes de cada llamada al LLM
Episodic (episódica)	episodic.py	Log JSONL de cada tool ejecutada en el turno: nombre, parámetros, resultado, timestamp. Un archivo por <code>run_id</code> . Permite saber qué hizo el agente en cada turno.	Al ejecutar y al fallar tools
Semantic (semántica)	semantic.py	ChromaDB con embeddings de casos aprendidos. Score híbrido: $\text{cosine_sim} \cdot (1-w) + \text{recency} \cdot w$ con <code>half_life_days=90</code> . Recupera contexto relevante de turnos pasados similares.	Al inicio del turno para enriquecer el prompt
Compressor	compressor.py	Cuando una tool falla, comprime el episodio completo del error y lo guarda en memoria semántica para que futuros reintentos no repitan el mismo error.	Al detectar error en ToolDispatcher (hilo paralelo)

- **Score híbrido semántico:** El sistema no solo recupera los casos más *similares* (similitud coseno) sino también los más *recientes*. La fórmula es **score = cosine * (1 - w) + recency * w**, con **half_life_days = 90** (un caso de hace 3 meses tiene la mitad de peso de recencia que uno de hoy). Esto evita que casos viejos desplacen a casos recientes más relevantes.

16.6 Integración con core.py

La función **obtener_respuesta_ia()** en **app/core.py** conserva todas las validaciones de seguridad (clientes nulos, modelo del canal, modo mantenimiento). Solo al final, en lugar de correr el bucle monolítico, instancia el router y el run:

Paso en core.py	Responsabilidad
Validar cliente_ia (Claude) y cliente_gemini	Si ambos son None: retorna "mantenimiento" antes de llegar al orquestador
Si el canal requiere Claude y no hay API key	Retorna "error de configuración" al usuario

Paso en core.py	Responsabilidad
router = LLMRouter(canal, claude_client, ...)	Crea el router con el proveedor primario del canal
agent_run = AgentRun(usuario_id, canal, router, tools_map, ...)	Instancia el estado del turno con el mapa de todas las ~32 herramientas
result = agent_run.execute(pregunta, messages, adjuntos)	Delega completamente; core.py no toca ningún loop de tools
return result.text, result.messages	Devuelve el texto y el historial actualizado igual que antes

16.7 Tests Corregidos y Validación (96/96)

La refactorización también corrigió 10 fallos de tests pre-existentes. Se documentan aquí los patrones aprendidos para evitar que reaparezcan:

Bug corregido	Causa raíz	Solución aplicada
JID vacío colisionaba con todos los grupos	"" == "" es True → cualquier JID vacío en .env coincidía con cualquier remitente	Envolver con bool(jid) and remote_jid == jid en app/routes.py
Migraciones SQLite rompen en DB nueva	Funciones de migración asumen tablas que no existen en una base de datos recién creada	Patrón _safe_migrate(): envolver cada migración en try/except sqlite3.OperationalError en init_db()
_add_col falla en tabla inexistente	PRAGMA table_info devuelve vacío → el ALTER TABLE falla	Verificar existencia de la tabla antes de consultar sus columnas
Mocks de posventa parchaban módulo incorrecto	Las funciones viven en app.meli_postventa_notif, no en webhook_meli	Importar y parchear app.meli_postventa_notif directamente
DB_PATH no cambiaba con setenv	Es constante de módulo fijada al importar; os.environ parchado después no la afecta	monkeypatch.setattr(tickets_db, "DB_PATH", str(path))
PUT /pasos retorna dict, no lista	La respuesta cambió a {"pasos": [...], "auto_resuelto": bool}	data["pasos"] if isinstance(data, dict) else data en el test
obtener_respuesta_ia mock rechazaba canal=	La función ahora acepta canal= como kwarg; el mock no lo hacía	lambda _msg, _sender, **_kw: (...) en el mock



Comandos de validación:

```
source venv/bin/activate && python3 -m pytest tests/ -q → 96/96
python3 -c "from app.agent.run import AgentRun; print('OK')"
```

AGENTE_AUDITORIA_SKIP_WA=1 python3 scripts/auditar_scripts_cron.py